RELATÓRIO DE COMPARAÇÃO ENTRE ALGORITMOS ITERATIVOS E RECURSIVOS

Introdução

O objetivo deste trabalho é compreender as diferenças entre algoritmos iterativos e recursivos, analisando desempenho, consumo de memória e vantagens/desvantagens. Para isso, foram implementados os algoritmos de cálculo de **fatorial** e **sequência de Fibonacci** em versões iterativas e recursivas.

Explicação dos Algoritmos

- **Iteração**: utiliza laços (for, while) para repetir instruções até atingir a condição desejada.
- **Recursão**: uma função chama a si mesma até atingir o caso base.

Exemplos:

- **Fatorial iterativo**: acumula o produto em um laço.
- **Fatorial recursivo**: chama a função até n == 1.
- **Fibonacci iterativo**: usa variáveis auxiliares para calcular os termos.
- **Fibonacci recursivo**: chama a função para n-1 e n-2.

IMPLEMENTAÇÃO EM C

```
#include <stdio.h>

// Fatorial iterativo

long long fatorial_iterativo(int n) {

   long long resultado = 1;

   for (int i = 1; i <= n; i++) {

      resultado *= i;

   }

   return resultado;

}

// Fatorial recursivo

long long fatorial_recursivo(int n) {

   if (n == 0 || n == 1) return 1;

   return n * fatorial_recursivo(n - 1);

}

// Fibonacci iterativo

long long fibonacci_iterativo(int n) {

   if (n == 0) return 0;
```

```
    if (n == 1) return 1;
    long long a = 0, b = 1, c;
    for (int i = 2; i <= n; i++) {
        c = a + b;
        a = b;
        b = c;
    }
    return b;
}
// Fibonacci recursivo
long long fibonacci_recursivo(int n) {
    if (n == 0) return 0;
    if (n == 1) return 1;
    return fibonacci_recursivo(n - 1) + fibonacci_recursivo(n - 2);
}
int main() {
    int n = 10;
    printf("Fatorial Iterativo de %d = %lld\n", n, fatorial_iterativo(n));
    printf("Fatorial Recursivo de %d = %lld\n", n, fatorial_recursivo(n));
    printf("Fibonacci Iterativo de %d = %lld\n", n, fibonacci_iterativo(n));
    printf("Fibonacci Recursivo de %d = %lld\n", n, fibonacci_recursivo(n));
    return 0;
}
```

RESULTADOS OBTIDOS

- Ambos os métodos produzem os mesmos resultados corretos.
- Para valores grandes de n, o **fatorial recursivo** pode causar *stack overflow*.
- O **Fibonacci recursivo** é extremamente lento para valores altos, pois repete cálculos desnecessários.

Análise de Complexidade

- **Fatorial iterativo**: (O(n)) tempo, (O(1)) memória.
- **Fatorial recursivo**: (O(n)) tempo, (O(n)) memória (pilha de chamadas).
- **Fibonacci iterativo**: (O(n)) tempo, (O(1)) memória.

- **Fibonacci recursivo**: (O(2^n)) tempo, (O(n)) memória.

VANTAGENS E DESVANTAGENS

- **Iteração**: mais eficiente em tempo e memória, mas menos elegante em alguns problemas.
- **Recursão**: código mais simples e próximo da definição matemática, mas pode ser ineficiente e consumir muita memória.

EXERCÍCIOS RESOLVIDOS

Exercício 1 – Stack Overflow em Recursão

Enunciado: Explique o que acontece se chamarmos fatorial_recursivo(100000).
Resposta: O programa provavelmente causará *stack overflow*, pois cada chamada recursiva ocupa espaço na pilha.

Exercício 2 – Fibonacci com Memoização

Enunciado: Reescreva o algoritmo recursivo de Fibonacci usando **memoização**.
Resposta: Guardar resultados já calculados em um vetor para evitar recomputações, reduzindo a complexidade para (O(n)).

```
long long memo[1000];

long long fibonacci_memo(int n) {

   if (n == 0) return 0;

   if (n == 1) return 1;

   if (memo[n] != -1) return memo[n];

   memo[n] = fibonacci_memo(n - 1) + fibonacci_memo(n - 2);

   return memo[n];

}
```

Exercício 3 – Soma Recursiva de Vetor

Enunciado: Crie uma função recursiva que calcule a soma dos elementos de um vetor.
Resposta:

```
int soma_recursiva(int v[], int n) {

   if (n == 1) return v[0];

   return v[n-1] + soma_recursiva(v, n-1);

}
```

Exercício 4 – Número de Chamadas Recursivas

Enunciado: Compare o número de chamadas recursivas em fibonacci_recursivo(10) e fibonacci_recursivo(20).
Resposta:

- Para n=10, são cerca de 177 chamadas.
- Para n=20, são mais de 13.000 chamadas.

Exercício 5 – Soma Iterativa de Dígitos

Enunciado: Implemente uma versão iterativa para calcular a soma dos dígitos de um número.
Resposta:

```
int soma_digitos(int n) {
    int soma = 0;
    while (n > 0) {
        soma += n % 10;
        n /= 10;
    }
    return soma;
}
```

Exercício 6 – Recursão em Árvores Binárias

Enunciado: Explique por que a recursão é mais adequada para percorrer árvores binárias do que a iteração.
Resposta: Estruturas hierárquicas se encaixam naturalmente em chamadas recursivas. Iteração exigiria estruturas auxiliares (pilhas/filas), tornando o código mais complexo.

Exercício 7 – MDC Recursivo (Euclides)

Enunciado: Escreva uma função recursiva que calcule o máximo divisor comum (MDC) usando o algoritmo de Euclides.
Resposta:

```
int mdc(int a, int b) {
    if (b == 0) return a;
    return mdc(b, a % b);
}
```